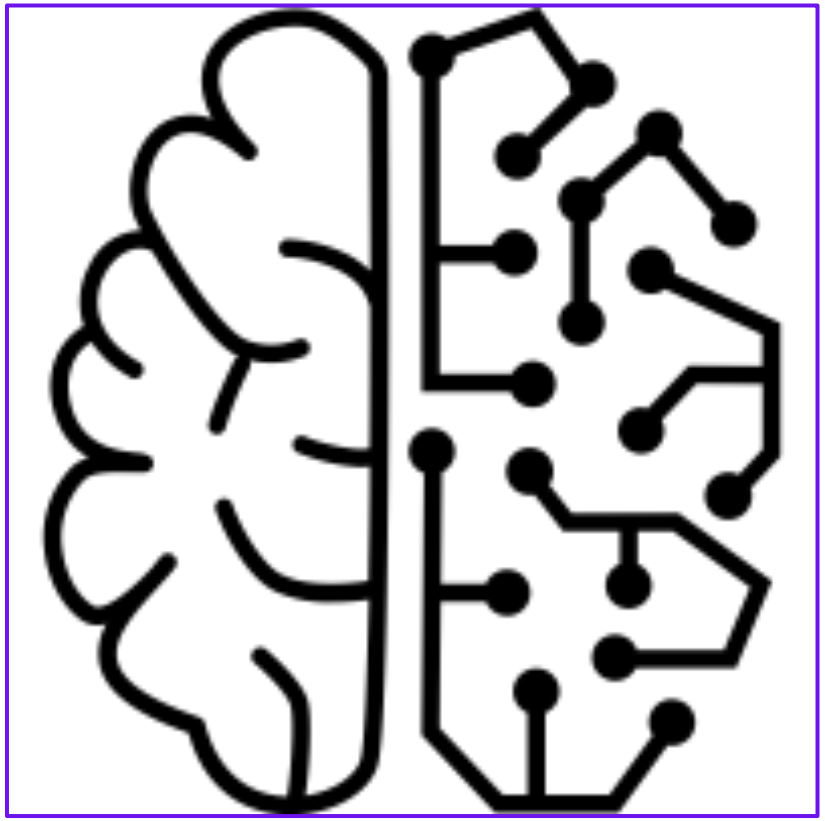

Application web pour visualiser les données

M2

NEXA Digital School

Compétences visées

- Introduction à Streamlit et structure de l'app
- Visualisations interactives avec Plotly
- Interface utilisateur pour les prédictions
- Design responsive et accessibilité

NEXA Digital School

Ressources

NEXA Digital School

Programme du Jour

- Matin (9h - 12h30)
  - Introduction à Streamlit et structure de l'app
  - Visualisations interactives avec Plotly
- Après-midi (13h30 - 17h00)
  - Interface utilisateur pour les prédictions
  - Design responsive et accessibilité

Objectif : Créer une application web fonctionnelle qui affiche les données et permet de faire des prédictions.

Interface Web avec Streamlit

NEXA Digital School

Récap Jour 1

- **Ce qu'on a construit hier**
  - Dataset nettoyé avec features engineerées
  - Modèle de ML entraîné (Régression Logistique)
  - Métriques évaluées (AUC ~0.75-0.80)
  - Biais détectés et corrigés

Aujourd'hui : De Jupyter à Production

Le problème : Notre modèle est dans un notebook, personne d'autre ne peut l'utiliser

La solution : Créer une application web avec Streamlit
- Interface graphique pour les utilisateurs
- Pas besoin d’etre expert en Python
- Déployable facilement

Qu'est-ce que Streamlit ?

Définition

Framework Python pour créer des applications web de data science en quelques lignes de code, sans HTML/CSS/JS.

Pourquoi Streamlit ?

- **Simple** : Du Python pur, pas de frontend
- **Rapide** : Prototyper en minutes
- **Interactif** : Widgets natifs (sliders, boutons, etc.)
- **Gratuit** : Open source + hébergement gratuit
- **Populaire** : Utilisé par des milliers d'entreprises

Cas d'usage

- Dashboards de ML/Data Science
- Outils d'exploration de données
- Démos de modèles IA
- Applications métier internes

Architecture d'une application Streamlit

Le fichier unique : `app.py`

```
import streamlit as st

# Configuration de la page
st.set_page_config(
    page_title="Loan Approval Predictor",
    page_icon="💰",
    layout="wide"
)
# Titre
st.title("🏦 Prédiction d'Approbation de Prêt")
# Contenu
st.write("Bienvenue dans notre application !")
```

Lancement de l'app

```
streamlit run app.py
```

L'app s'ouvre automatiquement dans le navigateur à http://localhost:8501

Les composants de base Streamlit

➢ Texte et titres

```
st.title("Titre principal")          # H1
st.header("Titre de section")        # H2
st.subheader("Sous-titre")           # H3
st.text("Texte simple")
st.markdown("**Markdown** avec *formatage*")
st.caption("Légende en petit")
st.code("print('Hello')", language='python')
```

➢ Données

```
st.dataframe(df)                # Tableau interactif
st.table(df.head())             # Tableau statique
st.json({"key": "value"})       # JSON formaté
st.metric("Accuracy", "78%", "+5%")  # Métrique avec delta
```

➢ Médias

```
st.image("logo.png")
st.audio("sound.mp3")
st.video("video.mp4")
```

Inputs utilisateur

```python
# Bouton
if st.button("Cliquez ici"):
    st.write("Bouton cliqué !")

# Slider
age = st.slider("Âge", min_value=18, max_value=100, value=30)

# Input numérique
income = st.number_input("Revenu mensuel", min_value=0, value=3000)

# Selectbox (menu déroulant)
education = st.selectbox("Niveau d'éducation", ["Graduate", "Not Graduate"])

# Checkbox
agree = st.checkbox("J'accepte les conditions")

# Radio buttons
married = st.radio("Statut marital", ["Célibataire", "Marié(e)"])

# Upload de fichier
uploaded_file = st.file_uploader("Choisir un fichier CSV")
```

Layout : Organiser l'interface

Colonnes

```
col1, col2, col3 = st.columns(3)

with col1:
    st.header("Colonne 1")
    st.write("Contenu gauche")

with col2:
    st.header("Colonne 2")
    st.write("Contenu centre")

with col3:
    st.header("Colonne 3")
    st.write("Contenu droite")
```

Sidebar (barre latérale)

```
st.sidebar.title("Options")
st.sidebar.selectbox("Choisir un modèle", ["Logistic
Regression", "Random Forest"])
st.sidebar.slider("Seuil de prédiction", 0.0, 1.0, 0.5)
```

Expander (section repliable)

```
with st.expander("Voir les détails"):
    st.write("Contenu caché par défaut")
    st.dataframe(df)
```

État et cache : optimisation

Le problème : réexécution complète

À chaque interaction, tout le script est réexécuté. Problème si on charge des données ou entraîne un modèle.

Solution 1 : Session State

```python
# Initialiser
if 'counter' not in st.session_state:
    st.session_state.counter = 0

# Incrémenter
if st.button("Incrémenter"):
    st.session_state.counter += 1

st.write(f"Compteur : {st.session_state.counter}")
```

Solution 2 : Cache

```python
@st.cache_data  # Pour les DataFrames, listes, etc.
def load_data():
    return pd.read_csv("loan_data.csv")

@st.cache_resource  # Pour les modèles ML, connexions DB
def load_model():
    return joblib.load("model.pkl")

# Ces fonctions ne s'exécutent qu'une fois
df = load_data()
model = load_model()
```

TP : Créer la structure de l'app

Créer la structure de l'application

Objectifs

- Créer un fichier app.py
- Configurer la page (titre, icône, layout wide)
- Créer 3 pages avec tabs :
  - "📊 Exploration des données"
  - "🤖 Prédiction"
  - "📈 Performance du modèle"
- Ajouter une sidebar avec :
  - Logo/titre de l'app
  - Sélecteur de modèle (Logistic Regression / Random Forest)
- Charger le dataset avec cache
- Charger le modèle avec cache
- Afficher le dataset dans l'onglet Exploration

VISUALISATIONS INTERACTIVES

NEXA Digital School

Matplotlib vs Plotly dans Streamlit

Matplotlib/Seaborn

Avantages : Simple, connu, rapide
Inconvénients : Statique, pas d'interaction

```
import matplotlib.pyplot as plt
import seaborn as sns

fig, ax = plt.subplots()
sns.histplot(df['ApplicantIncome'], ax=ax)
st.pyplot(fig)  # Afficher dans Streamlit
```

Plotly

Avantages : Interactif (zoom, hover, export), moderne.
Inconvénients : Un peu plus complexe

Recommandation : Utiliser Plotly pour les apps Streamlit

```
import plotly.express as px

fig = px.histogram(df, x='ApplicantIncome', title='Distribution des revenus')
st.plotly_chart(fig, use_container_width=True)
```

Visualisations essentielles avec Plotly

1. Histogramme (distribution)

```
import plotly.express as px

fig = px.histogram(
    df,
    x='ApplicantIncome',
    title='Distribution des revenus',
    labels={'ApplicantIncome': 'Revenu (€)'},
    color_discrete_sequence=['#636EFA']
)
st.plotly_chart(fig, use_container_width=True)
```

2. Box plot (outliers)

```
fig = px.box(
    df,
    y='LoanAmount',
    title='Distribution du montant du prêt',
    labels={'LoanAmount': 'Montant (€)'}
)
st.plotly_chart(fig)
```

NEXA Digital School

Visualisations (suite)

3. Bar chart (comparaison)

```
approval_by_education = df.groupby('Education')
['Loan_Status'].mean() * 100

fig = px.bar(
    x=approval_by_education.index,
    y=approval_by_education.values,
    title='Taux d\'approbation par niveau d\'éducation',
    labels={'x': 'Éducation', 'y': 'Taux d\'approbation (%)'},
    color=approval_by_education.values,
    color_continuous_scale='Viridis'
)
st.plotly_chart(fig)
```

```
fig = px.scatter(
    df,
    x='TotalIncome',
    y='LoanAmount',
    color='Loan_Status',
    title='Relation Revenu / Montant du prêt',
    labels={'TotalIncome': 'Revenu total (€)', 'LoanAmount': 'Montant (€)'},
    hover_data=['Credit_History']
)
st.plotly_chart(fig)
```

4. Scatter plot (corrélation)

Visualisations avancées

5. Heatmap (corrélation)

```python
import plotly.graph_objects as go

# Calculer la matrice de corrélation
corr = df.select_dtypes(include=['number']).corr()

fig = go.Figure(data=go.Heatmap(
    z=corr.values,
    x=corr.columns,
    y=corr.columns,
    colorscale='RdBu',
    zmid=0
))
fig.update_layout(title='Matrice de corrélation')
st.plotly_chart(fig)
```

6. Pie chart (proportions)

```python
loan_status_counts = df['Loan_Status'].value_counts()

fig = px.pie(
    values=loan_status_counts.values,
    names=['Approved', 'Rejected'],
    title='Répartition Approved / Rejected',
    color_discrete_sequence=['#00CC96', '#EF553B']
)
st.plotly_chart(fig)
```

Dashboard multi-colonnes

Layout professionnel avec métriques et graphiques

```python
# Métriques en haut
col1, col2, col3, col4 = st.columns(4)

with col1:
    st.metric("Total demandes", len(df))

with col2:
    approval_rate = df['Loan_Status'].mean() * 100
    st.metric("Taux d'approbation", f"{approval_rate:.1f}%")

with col3:
    avg_loan = df['LoanAmount'].mean()
    st.metric("Montant moyen", f"{avg_loan:.0f} €")

with col4:
    avg_income = df['TotalIncome'].mean()
    st.metric("Revenu moyen", f"{avg_income:.0f} €")

st.markdown("---")

# Graphiques en dessous
col1, col2 = st.columns(2)

with col1:
    fig1 = px.histogram(df, x='ApplicantIncome', title='Distribution revenus')
    st.plotly_chart(fig1, use_container_width=True)

with col2:
    fig2 = px.box(df, y='LoanAmount', title='Distribution montants')
    st.plotly_chart(fig2, use_container_width=True)
```

TP : Créer le dashboard d'exploration

Objectifs
Remplir l'onglet "📊 Exploration" avec :

- 4 métriques en haut :
  - Nombre total de demandes
  - Taux d'approbation global
  - Montant moyen des prêts
  - Revenu moyen
- Section "Distributions" (2 colonnes) :
  - Histogramme des revenus (ApplicantIncome)
  - Box plot des montants de prêt (LoanAmount)
- Section "Analyses" (2 colonnes) :
  - Bar chart : Taux d'approbation par éducation
  - Pie chart : Répartition Approved/Rejected
- Section "Corrélations" :
  - Heatmap de corrélation
- (Bonus) Ajouter des filtres dans la sidebar :
  - Slider pour filtrer par revenu min/max
  - Selectbox pour filtrer par niveau d'éducation

INTERFACE DE PRÉDICTION

Interface de prédiction interactive

Interface de prédiction interactive
Une interface où l'utilisateur peut :

- Saisir les caractéristiques d'un demandeur (revenu, prêt, historique, etc.)
- Cliquer sur "Prédire"
- Voir la décision du modèle (Approved/Rejected)
- Voir la probabilité de la décision
- Voir les facteurs qui ont influencé la décision

Design UX

- Formulaire clair avec labels explicites
- Validation des inputs
- Retour visuel immédiat (couleurs, icônes)
- Explications des résultats

Structure du formulaire de prédiction

Les features nécessaires
D'après notre modèle, on a besoin de :

- **ApplicantIncome** : Revenu du demandeur
- **CoapplicantIncome** : Revenu du co-demandeur
- **LoanAmount** : Montant demandé
- **Loan_Amount_Term** : Durée en mois
- **Credit_History** : Historique crédit (0/1)
- **Education** : Graduate / Not Graduate
- **Married** : Yes / No
- **Dependents** : Nombre de personnes à charge
- **Self_Employed** : Yes / No
- **Property_Area** : Urban / Semiurban / Rural

Utiliser st.form pour grouper les inputs

```
with st.form("prediction_form"):
    st.subheader("Informations du demandeur")

    col1, col2 = st.columns(2)

    with col1:
        applicant_income = st.number_input("Revenu mensuel (€)", min_value=0,
value=5000)
        # ... autres inputs

    with col2:
        coapplicant_income = st.number_input("Revenu co-demandeur (€)",
min_value=0, value=0)
        # ... autres inputs

    submitted = st.form_submit_button("🔮 Prédire l'approbation")
```

NEXA Digital School

Préparation des données pour la prédiction

Recréer les features engineerées

```python
if submitted:
    # Créer un dictionnaire avec les inputs
    input_data = {
        'ApplicantIncome': applicant_income,
        'CoapplicantIncome': coapplicant_income,
        'LoanAmount': loan_amount,
        'Loan_Amount_Term': loan_term,
        'Credit_History': credit_history,
        # ... autres features
    }

    # Créer un DataFrame
    input_df = pd.DataFrame([input_data])

    # Recréer les features engineerées (comme dans le TP3)
    input_df['TotalIncome'] = input_df['ApplicantIncome'] + input_df['CoapplicantIncome']
    input_df['LoanAmountToIncome'] = input_df['LoanAmount'] / (input_df['TotalIncome'] + 1)
    input_df['EMI'] = input_df['LoanAmount'] / input_df['Loan_Amount_Term']
    input_df['EMIToIncome'] = input_df['EMI'] / (input_df['TotalIncome'] + 1)
    input_df['Log_LoanAmount'] = np.log(input_df['LoanAmount'] + 1)
    input_df['Log_TotalIncome'] = np.log(input_df['TotalIncome'] + 1)
    input_df['Has_Coapplicant'] = (input_df['CoapplicantIncome'] > 0).astype(int)

    # Encoder les variables catégorielles (même encodage que l'entraînement)
    # ...
```

Faire la prédiction

Appeler le modèle

```python
# Normaliser (si Logistic Regression)
if model_choice == "Logistic Regression":
    input_scaled = scaler.transform(input_df)
    prediction = model.predict(input_scaled)[0]
    proba = model.predict_proba(input_scaled)[0]
else:
    prediction = model.predict(input_df)[0]
    proba = model.predict_proba(input_df)[0]

# Probabilité de la classe positive (Approved)
proba_approved = proba[1] * 100
proba_rejected = proba[0] * 100
```

Afficher le résultat

```python
st.markdown("---")
st.subheader("📊 Résultat de la prédiction")

if prediction == 1:
    st.success("✅ **PRÊT APPROUVÉ**")
    st.metric("Probabilité d'approbation", f"{proba_approved:.1f}%")
else:
    st.error("❌ **PRÊT REJETÉ**")
    st.metric("Probabilité de rejet", f"{proba_rejected:.1f}%")

# Barre de progression
st.progress(proba_approved / 100)
```

Expliquer la décision : Feature importance locale

Utiliser les coefficients du modèle

```python
if model_choice == "Logistic Regression":
    # Récupérer les coefficients
    feature_names = input_df.columns
    coefficients = model.coef_[0]

    # Calculer l'impact de chaque feature
    impacts = input_scaled[0] * coefficients

    # Créer un DataFrame
    impact_df = pd.DataFrame({
        'Feature': feature_names,
        'Impact': impacts
    }).sort_values('Impact', key=abs, ascending=False).head(5)

    st.subheader("🔍 Facteurs influents")
    st.write("Les 5 variables qui ont le plus influencé la décision :")

    fig = px.bar(
        impact_df,
        x='Impact',
        y='Feature',
        orientation='h',
        color='Impact',
        color_continuous_scale='RdYlGn',
        title='Contribution de chaque variable'
    )
    st.plotly_chart(fig)
```

Validation et gestion des erreurs

Valider les inputs

```python
# Avant de prédire, vérifier la cohérence
errors = []

if applicant_income < 1000:
    errors.append("⚠️ Le revenu du demandeur semble très bas")

if loan_amount > applicant_income * 100:
    errors.append("⚠️ Le montant du prêt est très élevé par rapport au revenu")

if loan_amount / loan_term > applicant_income:
    errors.append("⚠️ La mensualité dépasse le revenu mensuel")

if errors:
    st.warning("Points d'attention :")
    for error in errors:
        st.write(error)
    st.write("La prédiction reste possible mais les résultats peuvent être peu fiables.")
```

Gérer les exceptions

```python
try:
    prediction = model.predict(input_scaled)[0]
except Exception as e:
    st.error(f"❌ Erreur lors de la prédiction : {str(e)}")
    st.write("Vérifiez que toutes les données sont correctement renseignées.")
```

NEXA Digital School

TP : Créer l'interface de prédiction

Créer l'interface de prédiction

Objectifs
Remplir l'onglet "🤖 Prédiction" avec :

- Formulaire en 2 colonnes avec tous les inputs nécessaires :
  - Colonne 1 : Revenus, montant, durée
  - Colonne 2 : Historique crédit, éducation, statut marital, etc.
- Bouton "Prédire" qui :
  - Crée les features engineerées
  - Encode correctement les variables
  - Appelle le modèle
  - Affiche le résultat (Success/Error)
  - Affiche la probabilité
  - Affiche une barre de progression
- Section "Explication" :
  - Top 5 des features influentes (bar chart horizontal)
- (Bonus) Ajouter des validations et warnings

RESPONSIVE DESIGN & ACCESSIBILITÉ

Objectif : Performance du modèle

Ce qu'on va afficher dans l'onglet 3

- Métriques globales (Accuracy, Precision, Recall, F1, AUC)
- Matrice de confusion
- Courbe ROC
- Feature importance globale
- Distribution des probabilités

Pourquoi ? Transparence, confiance, audit

Charger les métriques pré-calculées

Charger les métriques pré-calculées

```python
# Charger le test set
X_test = pd.read_csv("X_test.csv")
y_test = pd.read_csv("y_test.csv")

# Prédire
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]

# Calculer les métriques
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, roc_auc_score

accuracy = accuracy_score(y_test, y_pred)
# ...
```

Option 2 : Charger des métriques pré-calculées

```python
import json
# Sauvegarder les métriques une fois (lors de l'entraînement)
metrics = {
    'accuracy': 0.78,
    'precision': 0.82,
    'recall': 0.89,
    'f1': 0.85,
    'auc': 0.82
}
with open('metrics.json', 'w') as f:
    json.dump(metrics, f)

# Charger dans Streamlit
with open('metrics.json', 'r') as f:
    metrics = json.load(f)
```

Recommandation : Option 2 (plus rapide)

Afficher les métriques

Cartes de métriques

```python
st.subheader("📊 Métriques de performance")

col1, col2, col3, col4, col5 = st.columns(5)

with col1:
    st.metric("Accuracy", f"{metrics['accuracy']:.2%}")

with col2:
    st.metric("Precision", f"{metrics['precision']:.2%}")

with col3:
    st.metric("Recall", f"{metrics['recall']:.2%}")

with col4:
    st.metric("F1-Score", f"{metrics['f1']:.2%}")

with col5:
    st.metric("AUC", f"{metrics['auc']:.3f}")

# Explications
with st.expander("ℹ️ Que signifient ces métriques ?"):
    st.write("""
    - **Accuracy** : Pourcentage de prédictions correctes
    - **Precision** : Parmi les prêts approuvés, % vraiment solvables
    - **Recall** : Parmi les prêts solvables, % correctement détectés
    - **F1-Score** : Moyenne harmonique de Precision et Recall
    - **AUC** : Aire sous la courbe ROC (0.5 = aléatoire, 1.0 = parfait)
    """)
```

Afficher la matrice de confusion

Créer une heatmap interactive

```python
import plotly.figure_factory as ff

# Matrice de confusion (pré-calculée)
cm = [[50, 10], [5, 60]]  # [[TN, FP], [FN, TP]]

# Labels
x = ['Rejected', 'Approved']
y = ['Rejected', 'Approved']

# Créer la heatmap
fig = ff.create_annotated_heatmap(
    cm,
    x=x,
    y=y,
    colorscale='Blues',
    showscale=True
)

fig.update_layout(
    title='Matrice de confusion',
    xaxis_title='Prédiction',
    yaxis_title='Vrai label'
)

st.plotly_chart(fig, use_container_width=True)

# Interprétation
st.write("""
- **True Negatives (TN)** : {} - Correctement rejetés
- **False Positives (FP)** : {} - Faussement approuvés ⚠️
- **False Negatives (FN)** : {} - Faussement rejetés ⚠️
- **True Positives (TP)** : {} - Correctement approuvés
""".format(cm[0][0], cm[0][1], cm[1][0], cm[1][1]))
```

NEXA Digital School

Afficher la courbe ROC

Plotly pour une courbe interactive

```python
# Pré-calculer fpr, tpr (ou charger depuis CSV)
fpr = [0, 0.1, 0.2, 0.5, 1.0]
tpr = [0, 0.6, 0.8, 0.95, 1.0]
auc = 0.82

fig = go.Figure()

# Courbe ROC
fig.add_trace(go.Scatter(
    x=fpr, y=tpr,
    mode='lines',
    name=f'ROC (AUC = {auc:.3f})',
    line=dict(color='darkorange', width=2)
))

# Ligne de référence (aléatoire)
fig.add_trace(go.Scatter(
    x=[0, 1], y=[0, 1],
    mode='lines',
    name='Random (AUC = 0.5)',
    line=dict(color='navy', width=2, dash='dash')
))

fig.update_layout(
    title='Courbe ROC (Receiver Operating Characteristic)',
    xaxis_title='False Positive Rate',
    yaxis_title='True Positive Rate',
    xaxis=dict(range=[0, 1]),
    yaxis=dict(range=[0, 1]),
    showlegend=True
)

st.plotly_chart(fig, use_container_width=True)

st.info("""
💡 **Interprétation** : Plus la courbe s'éloigne de la diagonale, meilleur est le modèle.
Un AUC de {:.2f} indique un {} modèle.
""".format(auc, "très bon" if auc >= 0.8 else "bon"))
```

NEXA Digital School

Feature Importance globale

Top 10 des features les plus importantes

```python
# Récupérer les importances
if model_choice == "Logistic Regression":
    importances = np.abs(model.coef_[0])
    feature_names = model.feature_names_in_
else:
    importances = model.feature_importances_
    feature_names = model.feature_names_in_

# Créer un DataFrame
importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values('Importance', ascending=False).head(10)

# Visualiser
fig = px.bar(
    importance_df,
    x='Importance',
    y='Feature',
    orientation='h',
    title='Top 10 des features les plus importantes',
    color='Importance',
    color_continuous_scale='Viridis'
)

st.plotly_chart(fig, use_container_width=True)

st.write("""
💡 Ces variables ont le plus d'impact sur les prédictions du modèle.
Typiquement, l'historique de crédit et les ratios financiers sont les plus importants.
""")
```

NEXA Digital School

Adapter l'interface aux petits écrans

Problème : Par défaut, Streamlit est responsive, mais on peut optimiser.
Bonnes pratiques

```
# 1. Utiliser use_container_width=True
st.plotly_chart(fig, use_container_width=True)  # ✅
st.plotly_chart(fig)  # ❌ Largeur fixe

# 2. Colonnes adaptatives
# Sur grand écran : 3 colonnes
# Sur petit écran : 1 colonne (automatique avec Streamlit)
col1, col2, col3 = st.columns(3)

# 3. Pas de largeurs fixes en pixels
st.markdown("<div style='width: 800px'>", unsafe_allow_html=True)  # ❌
# Laisser Streamlit gérer

# 4. Tester sur mobile
# Astuce : Ctrl+Shift+M dans Chrome pour mode mobile
```

Tester le responsive

- Réduire la fenêtre du navigateur
- Vérifier que les colonnes s'empilent
- Vérifier que les graphiques s'adaptent
- Vérifier que le texte reste lisible

Accessibilité : Rappels WCAG

Les points clés vus hier

- **Contraste** : Texte lisible sur fond
- **Alt text** : Descriptions pour images
- **Labels** : Inputs avec labels clairs
- **Ordre logique** : Navigation au clavier
- **Messages d'erreur** : Explicites et compréhensibles

Implémentation

```python
# ✅ BON : Label clair
st.number_input("Revenu mensuel net en euros", min_value=0)

# ❌ MAUVAIS : Label cryptique
st.number_input("Rev.", min_value=0)

# ✅ BON : Message d'erreur explicite
if income < 0:
    st.error("⚠️ Le revenu ne peut pas être négatif. Veuillez saisir un montant positif.")

# ❌ MAUVAIS : Message technique
if income < 0:
    st.error("Error: ValueError at line 42")

# ✅ BON : Couleurs + texte
st.success("✅ Prêt approuvé")  # Vert + texte

# ❌ MAUVAIS : Couleur seule
st.markdown("<span style='color: green;'>●</span>", unsafe_allow_html=True)
```

Style et thème Streamlit

Personnaliser les couleurs

Créer un fichier .streamlit/config.toml :

```toml
[theme]
primaryColor = "#FF4B4B"        # Rouge/rose
backgroundColor = "#FFFFFF"      # Blanc
secondaryBackgroundColor = "#F0F2F6"  # Gris clair
textColor = "#262730"           # Noir
font = "sans serif"

[server]
headless = true
port = 8501
```

Custom CSS (optionnel)

```python
st.markdown("""
<style>
    .stButton>button {
        width: 100%;
        height: 3em;
        font-size: 18px;
    }
    h1 {
        color: #FF4B4B;
    }
</style>
""", unsafe_allow_html=True)
```

TP : Finaliser l'application

Métriques d'évaluation

Objectifs

- **Remplir l'onglet "📈 Performance"** avec :
  - 5 métriques en cartes
  - Matrice de confusion (heatmap)
  - Courbe ROC
  - Feature importance (top 10)
  - Expanders avec explications
- Améliorer l'UX globale :
  - Ajouter des émojis aux titres
  - Ajouter des st.info() / st.warning() pour guider l'utilisateur
  - Vérifier que tous les graphiques ont use_container_width=True
  - Ajouter une section "À propos" en bas avec :
    - Explication du projet
    - Méthodologie (RGPD, éthique)
    - Contact
- Tester l'accessibilité :
  - Tous les inputs ont des labels clairs
  - Messages d'erreur explicites
  - Contraste suffisant
  - Navigation logique
- (Bonus) Ajouter un footer avec :
  - Version de l'app / Date de dernière mise à jour / Lien vers la documentation